Vulnerabilidades

© **Universidad de Palermo** | Prohibida la reproducción total o parcial de imágenes y textos.

Introducción a Metasploit

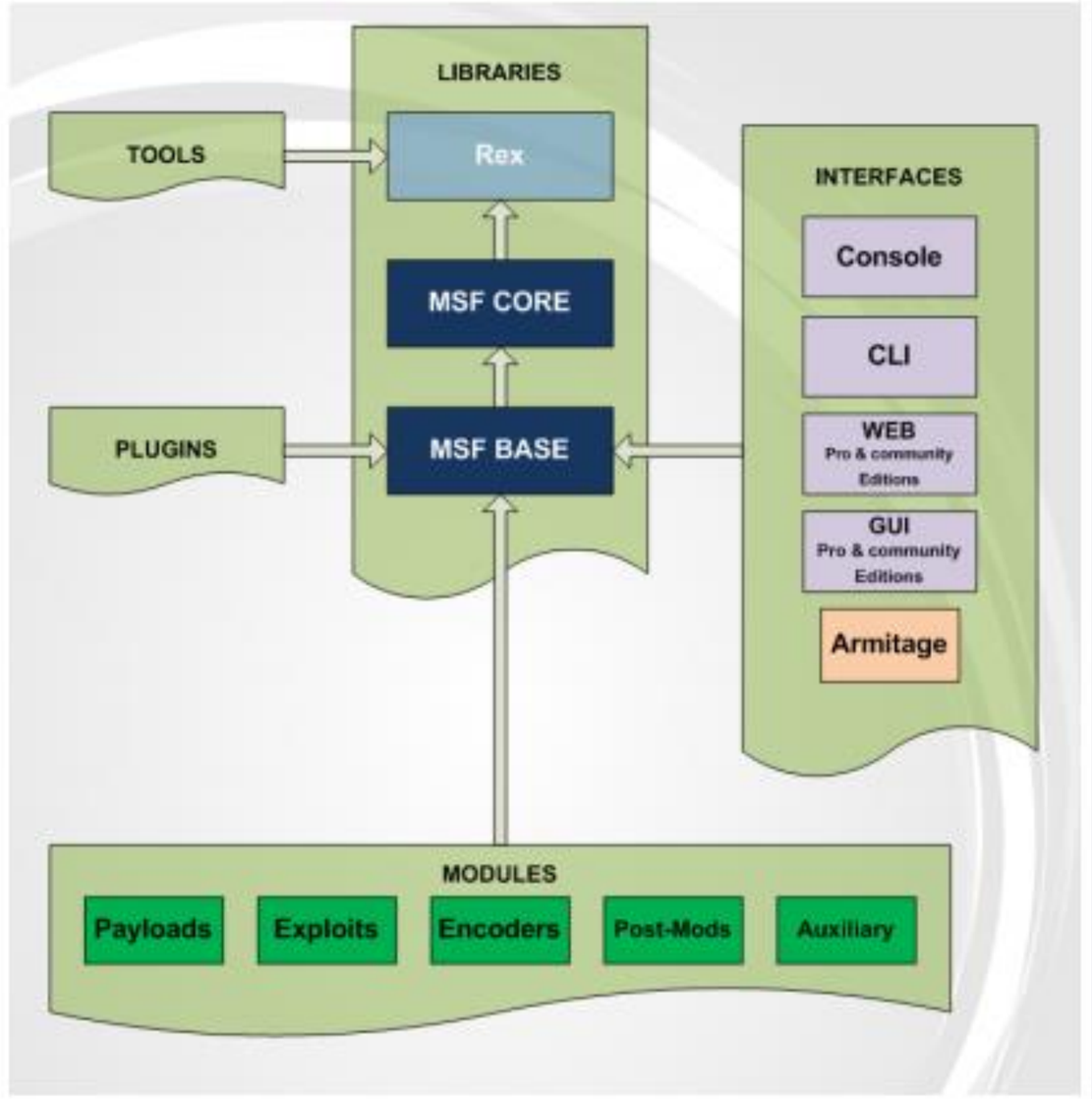
Metasploit es un framework multiplataforma escrito en Ruby que abstrae las tareas típicas de una intrusión, ofreciendo un esquema modular donde combinar e integrar distintos tipos de exploits y herramientas de acceso y control de equipos y servicios comprometidos.

Incluye también módulos adicionales para las fases de rastreo y enumeración.

UP Universidad de Palermo ©

Arquitectura de Metasploit

Metasploit sigue una arquitectura modular, organizada alrededor de un núcleo que estructura la aplicación y ofrece las funcionalidades básicas.



Fuente: https://kalilinux.foroactivo.com/t48-tutorial-metasploit-para-kali-linux-parte-1-introduccion

UP Universidad de Palermo ©

Exploits

- Piezas de código que explotan una vulnerabilidad concreta que permite un acceso no previsto.
- Suelen ser específicas del sistema operativo y de la versión concreta del servicio, aunque hay algunos exploits independientes de la plataforma.
- Su uso principal es como "vector" para la inyección de un payload específico que ofrezca al atacante algún tipo de acceso y/o control del equipo comprometido.

UP Universidad de Palermo ©

Payloads

- Piezas de código que permiten algún tipo de acceso o control sobre un equipo que ha sido comprometido mediante la explotación de alguna vulnerabilidad.
- Suelen ser específicos del sistema operativo, aunque algunos basados en Java o lenguajes de Script son independientes de la plataforma.
- Uno de los payloads más potentes que ofrece Metasploit es Meterpreter.
- Se trata de un payload que ofrece un intérprete de comandos en el sistema comprometido, complementado con una serie de comandos específicos que soportan tareas típicas de una intrusión (recopilación de información del sistema comprometidos, keylogger, ocultación de rastros, etc.).

UP Universidad de Palermo ©

Auxiliary

- Módulos auxiliares que automatizan tareas complementarias empleadas habitualmente en test de intrusión.
- Fundamentalmente se trata de diversos tipos de escáneres:
  - Escáner de puertos genéricos o
  - Escáneres específicos para aplicaciones/servicios concretos.
- También se proveen módulos para:
  - Recopilar credenciales de acceso basados en diccionarios o romper contraseñas.
  - Enumeradores de directorios.
  - Herramientas para recopilación de información de red.
  - Una colección de fuzzers que generan cadenas de entrada aleatorias con las que detectar posibles vulnerabilidades en la validación de entradas.

UP Universidad de Palermo ©

Post

- Piezas de código específicas de cada arquitectura o aplicación que automatizan tareas relativas al mantenimiento, extensión y/o ocultación del acceso a equipos comprometidos.

- Fundamentalmente ofrecen funcionalidades para recopilar información del sistema comprometidos (servicios, usuarios, archivos) para escalar privilegios obteniendo credenciales de administrador o para ocultar el rastro de la explotación.

UP Universidad de Palermo ©

Nops

Módulos complementarios usados para generar distintos tipos de códigos NOP (No operation) para diferentes arquitecturas y CPUs a utilizar en el código de los exploits y sus respectivos payloads.

UP Universidad de Palermo ©

Encoders

- Módulos complementarios utilizados para ofuscar y ocultar el código de los exploits y sus respectivos payloads empleando diversos tipos de codificación.

- Son un mecanismo de evasión para evitar la detección del ataque por parte de IDS (sistemas de detección de intrusiones) o antivirus.

UP Universidad de Palermo ©

Msfconsole

- Hay diferentes interfaces para el framework Metasploit, cada uno con sus ventajas y desventajas.
- Como tal, no hay una interfaz perfecta para usar con MSF, aunque msfconsole es la única forma soportada para acceder a la mayoría de las características del framework.
- Es por eso que, de ahora en adelante, vamos a hablar solamente de la utilización de msfconsole.

UP Universidad de Palermo ©

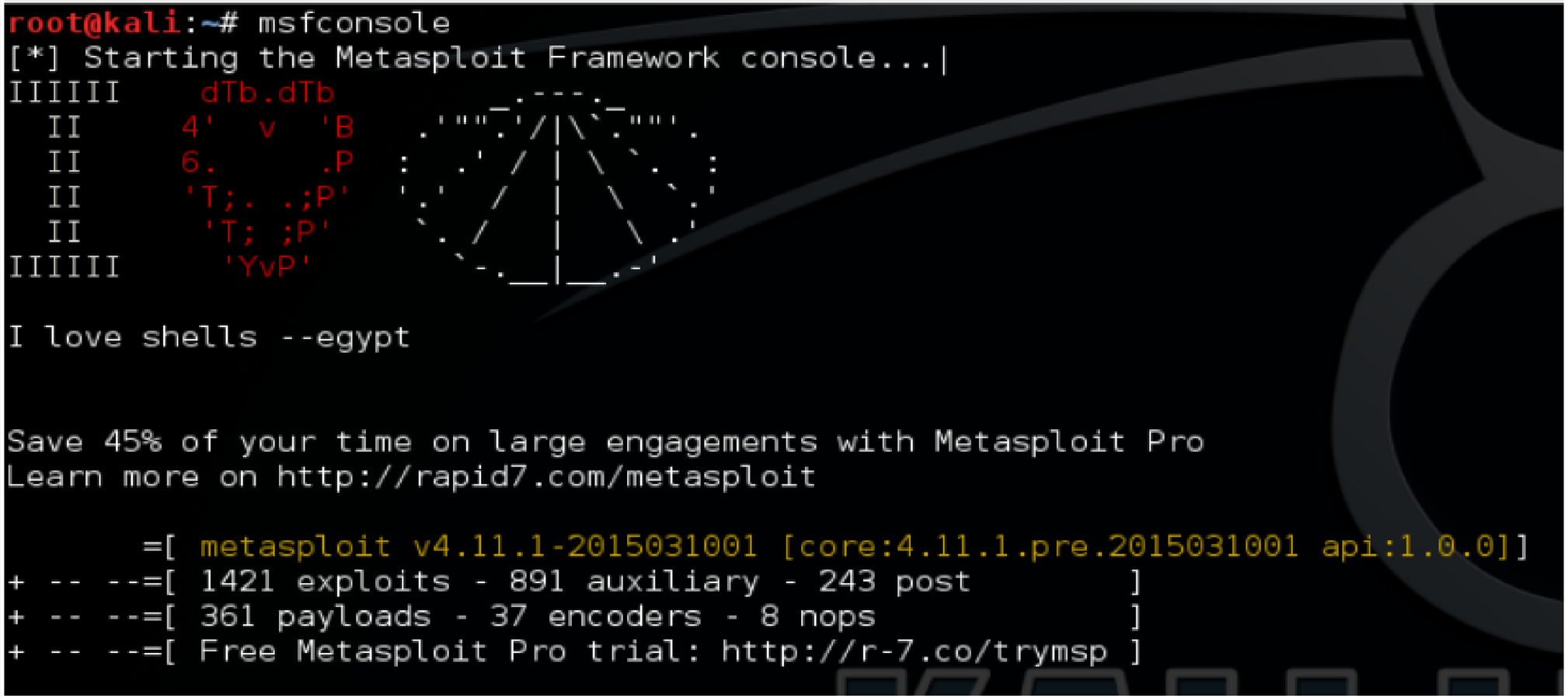
Inicialización de BD

Manage the metasploit framework database

```
msfdb init     # start and initialize the database
msfdb reinit   # delete and reinitialize the database
msfdb delete   # delete database and stop using it
msfdb start    # start the database
msfdb stop     # stop the database
msfdb status   # check service status
msfdb run      # start the database and run msfconsole
```

UP Universidad de Palermo ©

Pantalla de bienvenida de msfconsole

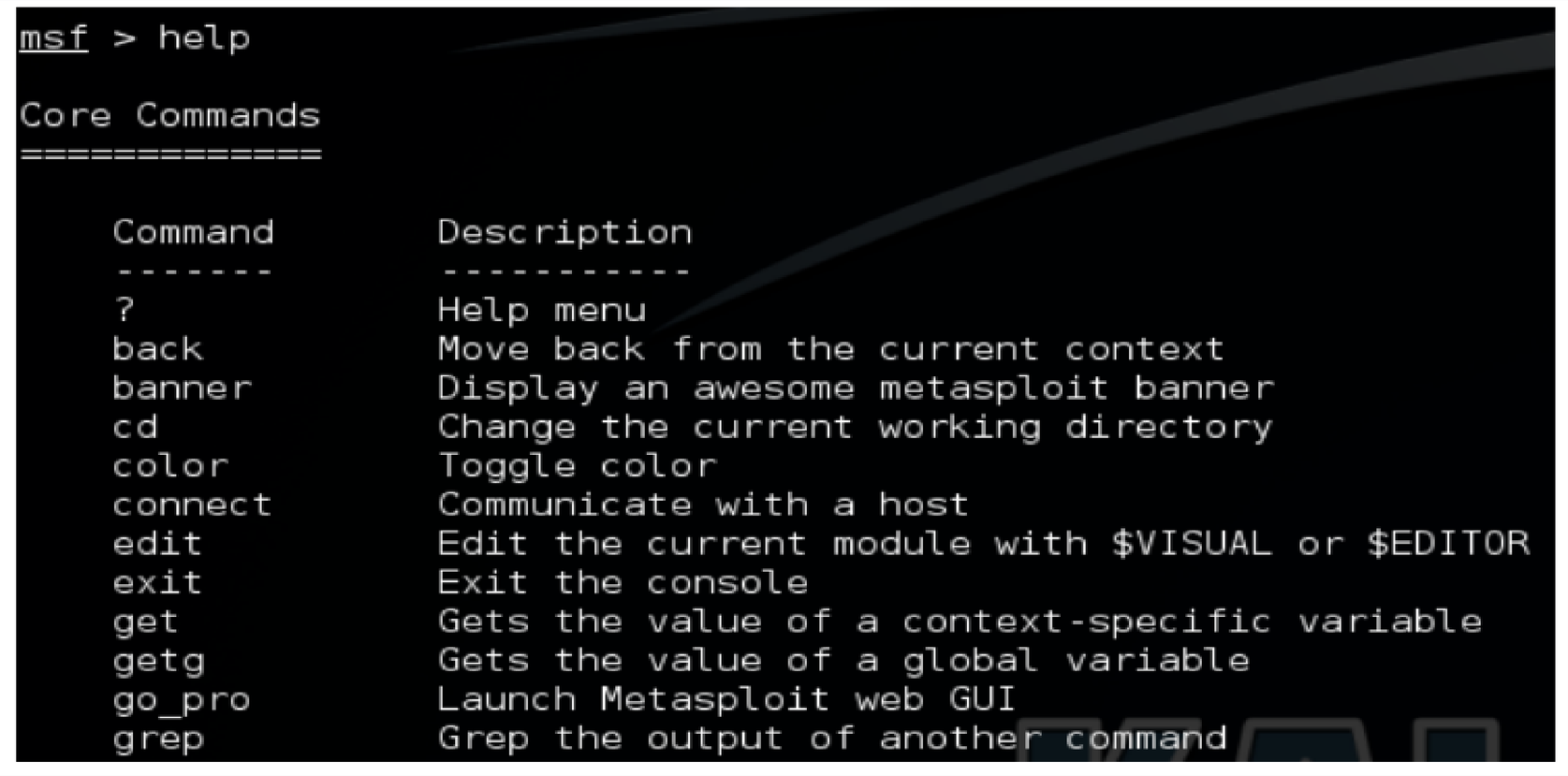


Pantalla de bienvenida de msfconsole

Fuente: Pantalla de bienvenida de msfconsole [captura de pantalla]

Obteniendo ayuda

Escribiendo "help" o "?" en el command promp de msf, mostrará una lista de los comandos disponibles junto una descripción de su uso.



Fuente: Comando “help” en ejecución del programa msfconsole [captura de pantalla]

Autocompletado

- Una de las características más útiles de msfconsole es la completación con la tecla 'Tab'.
- Con la gran variedad de módulos disponibles, puede ser difícil recordar el nombre exacto y ruta de algún módulo en particular que se quiera usar.
- Como en la mayoría de los shells, escribiendo lo que sabés y presionando "Tab“ mostrará una lista de las opciones disponibles o se autocompletará la palabra si hay una sola opción.

UP Universidad de Palermo ©

Comando “show”

Introduciendo "show" en el prompt de msfconsole mostrará cada módulo disponible en Metasploit.

Hay una variedad de comandos "show" que podemos utilizar, pero los que vamos a utilizar con más frecuencia son:

- "show auxiliary"
- "show exploits"
- "show payloads"

Como podemos ver, hay muchos payloads disponibles.

Afortunadamente cuando estamos utilizando un exploit determinado, usando **"show payloads"** solo veremos los payloads que son compatibles para ese particular exploit.

Por ejemplo, si es un exploit para Windows, no se mostrarán los payloads para Linux.

Si hemos seleccionado un módulo específico, podemos utilizar el comando "show options" para mostrar las opciones disponibles y/o requeridas para ese módulo en específico.

Si no estamos seguros de si un sistema operativo es vulnerable a un particular exploit, podemos ejecutar el comando **"show targets"** dentro del contexto de un módulo de exploit para ver qué objetivos están soportados.

UP Universidad de Palermo ©



Fuente: Cargando un exploit. [captura de pantalla].

Comando “search”

El msfconsole incluye una funcionalidad de búsqueda basada en expresiones regulares.

Si tenemos alguna idea general de lo que estamos buscando, podemos buscarlo con "search".

```
msf exploit(ms08_067_netapi) > help search
Usage: search [keywords]

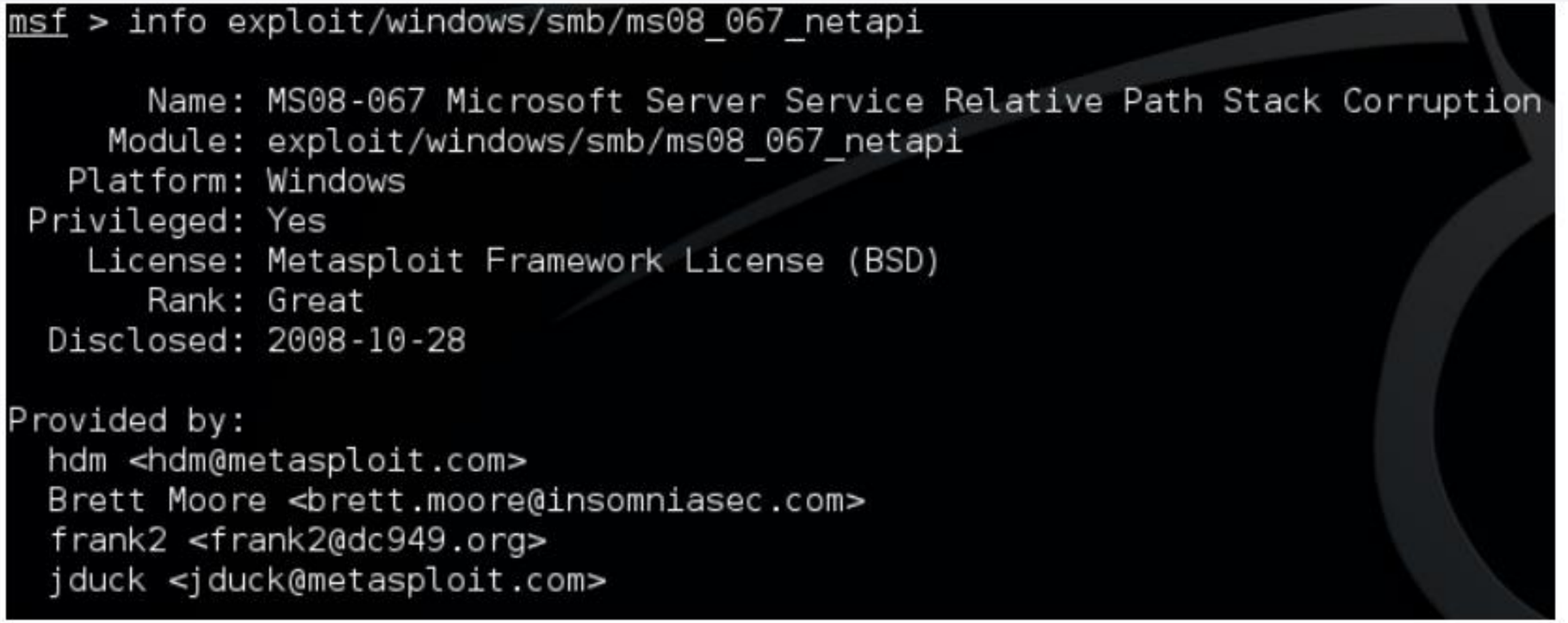
Keywords:
  app         :  Modules that are client or server attacks
  author      :  Modules written by this author
  bid         :  Modules with a matching Bugtraq ID
  cve         :  Modules with a matching CVE ID
  edb         :  Modules with a matching Exploit-DB ID
  name        :  Modules with a matching descriptive name
  osvdb       :  Modules with a matching OSVDB ID
  platform    :  Modules affecting this platform
  ref         :  Modules with a matching ref
  type        :  Modules of a specific type (exploit, auxiliary, or post)

Examples:
  search cve:2009 type:exploit app:client
```

Fuente: Ejecución del comando “search” en el programa msfconsole. [captura de pantalla].

Comando “info”

El comando "info" proporciona información detallada sobre un determinado módulo incluyendo todas las opciones, objetivos, y otras informaciones.



Información del exploit ms08_067_netapi

Fuente: Cargando un exploit desde la consola de comando de Metasploit [captura de pantalla].

UP Universidad de Palermo ©

Comandos

Comando "use"

Cuando se ha decidido por un módulo en particular, utilizamos el comando "use" para seleccionarlo.

Comando "connect"

Utilizando el comando "connect" con una dirección IP y un número de puerto, podemos conectarnos a un host remoto desde msfconsole igual como si usáramos netcat o telnet.

Comando "set“

Es usado para configurar las opciones del módulo que actualmente estamos utilizando.

UP Universidad de Palermo ©

Comandos

Comando "check"

No hay muchos exploits que lo soporten, pero hay una opción de "check" la cual comprueba si el objetivo es vulnerable a un exploit en particular en lugar de explotarla.

Variables globales

- Con el fin de no escribir mucho durante un pentest, podemos establecer variables globales dentro de msfconsole.
- Podemos hacer esto con el comando "setg".
- Una vez que se ha establecido, la podemos utilizar en muchos exploits y módulos auxiliares a nuestro gusto.
- También la podemos guardar para usarlo la próxima vez que utilicemos msfconsole.
- Sin embargo, ciertos datos no son guardados, así que siempre debemos revisar las opciones definidas antes de usar "run" o "exploit".
- Por el contrario, podemos usar el comando "unsetg" para eliminar una variable global.

Metasploit no es sensible a las mayúsculas (es lo mismo definir una variable en mayúsculas que en minúsculas).

UP Universidad de Palermo ©

Comandos

Comando "exploit/run“

Para lanzar un exploit, podemos usar el comando "exploit" mientras que si usamos un módulo auxiliar, el uso correcto es "run" aunque "exploit" funciona también.

Comando "back"

- Cuando hayamos terminado de usar un módulo, o si seleccionamos el módulo equivocado, podemos usar el comando "back" para salir de ese contexto.
- Esto, sin embargo no es requerido, porque podemos cambiar de módulo estando dentro de otro.
- Como recordatorio, las variables son cambiadas si son establecidas globalmente.

UP Universidad de Palermo ©

Comandos

Comando "resource"

- Algunos ataques como Karmetasploit usan archivos adicionales que se pueden cargar a través de msfconsole usando el comando "resource".
- Estos archivos son unos scripts básicos para msfconsole.
- Ejecutan los comandos del archivo en secuencia.

Comando "irb"

Si ejecutamos el comando "irb", cambiará al modo de edición de scripts en ruby, donde podremos usar comandos y crear scripts en el momento.

UP Universidad de Palermo ©

Payloads

Metasploit contiene muchos tipos diferentes de payloads, cada uno cumple un rol único dentro del framework.

Echemos un vistazo a los varios tipos de payloads disponibles para hacernos una idea de cuándo debería utilizarse cada uno.

UP Universidad de Palermo ©

Payloads

En Línea (No Efectuado, Non Staged)

Un payload único conteniendo el exploit y el código shell (shellcode) completo para la tarea seleccionada.

Los payloads en línea son más estables por diseño que sus contrapartes porque contienen todo en uno.
Sin embargo, algunos exploits no soportan el tamaño resultante de estos payloads.

Efectuado (Staged)

Los payloads staged trabajan conjuntamente con los payloads stage para llevar a cabo una tarea específica.

Un staged establece un canal de comunicación entre el atacante y la víctima y lee en un payload efectuado lo que debe ejecutar en el host remoto.

Meterpreter

Es el diminutivo de MetaInterpreter, y es un payload avanzado, multifacético, que opera por medio de inyección en librerías dinámicas (dll).

El Meterpreter reside completamente en la memoria del host remoto y no deja rastros en el disco duro, haciéndolo muy difícil de detectar con técnicas forenses convencionales.

Los scripts y plugins se pueden cargar y descargar dinámicamente según se necesitan.

El desarrollo de Meterpreter es muy poderoso y en constante evolución.

PassiveX

Es un payload que puede ayudar a evitar cortafuegos con la salida restringida.

Esto lo realiza utilizando un control ActiveX para crear una instancia oculta de Internet Explorer.

Utilizando el nuevo control ActiveX, se comunica con el atacante por medio de peticiones y respuestas HTTP.

UP Universidad de Palermo ©

Payloads

NoNX

El bit NX (No eXecute) es una característica incorporada en algunas CPUs para prevenir que se ejecute código en ciertas áreas de la memoria,

En Windows, NX esta implementado como Data Execution Prevention (DEP), Prevención de Ejecución de Datos.

Los payloads NoNX de Metasploit están diseñados para evitar DEP.

IPv6

Los payloads IPv6 de Metasploit, como indica su nombre, han sido construidos para funcionar sobre redes IPv6.

UP Universidad de Palermo ©

Payloads

Ord

Los payloads ordinales son payloads staged basados en Windows que tienen distintas ventajas y desventajas.
Las ventajas son que trabajan en cualquier sabor y lenguaje de Windows a partir de Windows 9x sin la definición explícita de una dirección de retorno.
También son extremadamente pequeños.
Sin embargo, dos desventajas muy específicas hacen que no sean la opción por defecto.
La primera reside en el hecho de que ws2_32.dll se carga en el proceso que está siendo explotado antes de la explotación.
La segunda es que es un poco menos estable que los otros stageds.

Inyección DLL Reflectiva

La Inyección DLL Reflectiva es una técnica mediante la cual un staged payload es inyectado en un proceso corriendo en la memoria del host comprometido, sin tocar nunca el disco duro del host.

Ambos payloads VNC y Meterpreter hacen uso de la inyección DLL reflectiva.

Vulnerabilidades de una máquina

Para listar las posibles vulnerabilidades de una máquina:

use post/multi/recon/local_exploit_suggester

show options

exploit

UP Universidad de Palermo ©

Bind vs. Reverse

En muchos de los payloads disponibles podremos observar las palabras ‘bind’ o ‘reverse’.

Siempre veremos una o la otra, nunca las dos juntas.

Esto hace referencia al método de conexión que utiliza el payload para comunicarse con el atacante.

- **Bind:** abren un puerto (lo ponen en ‘escucha’) y esperan la conexión por parte del atacante.
- **Reverse:** hace que la víctima inicie una conexión saliente hacia el sistema del atacante.

Esta diferencia, aunque parezca menor, es de suma importancia al momento de seleccionar un payload. Su elección va a depender de cómo puedan conectarse los sistemas involucrados (víctima y atacante).

UP Universidad de Palermo ©

Comandos útiles

A continuación tenemos una lista de comandos que van a ser de utilidad una vez que hayamos tomado el control de un sistema remoto y tengamos una sesión de Meterpreter.

UP Universidad de Palermo ©

PS

```
meterpreter > ps

Process List
============

 PID   PPID  Name                Arch  Session     User
 ---   ----  ----                ----  -------     ----
 0     0     [System Process]          4294967295
 4     0     System              x86   0           NT AUTHORITY\SYSTEM
 368   4     smss.exe            x86   0           NT AUTHORITY\SYSTEM
 516   1480  VBoxTray.exe        x86   0           COYOTE\Beep
 532   1480  wscript.exe         x86   0           COYOTE\Beep
 544   1048  wscntfy.exe         x86   0           COYOTE\Beep
 584   368   csrss.exe           x86   0           NT AUTHORITY\SYSTEM
 608   368   winlogon.exe        x86   0           NT AUTHORITY\SYSTEM
 652   608   services.exe        x86   0           NT AUTHORITY\SYSTEM
 664   608   lsass.exe           x86   0           NT AUTHORITY\SYSTEM
 788   652   alg.exe             x86   0           NT AUTHORITY\LOCAL SERVICE
 820   652   VBoxService.exe     x86   0           NT AUTHORITY\SYSTEM
 864   652   svchost.exe         x86   0           NT AUTHORITY\SYSTEM
 952   652   svchost.exe         x86   0           NT AUTHORITY\NETWORK SERVICE
 1048  652   svchost.exe         x86   0           NT AUTHORITY\SYSTEM
 1100  652   svchost.exe         x86   0           NT AUTHORITY\NETWORK SERVICE
 1136  652   svchost.exe         x86   0           NT AUTHORITY\LOCAL SERVICE
 1292  532   ZRwqoCxAdYwgnNs.exe x86   0           COYOTE\Beep
 1348  996   cmd.exe             x86   0           COYOTE\Beep
 1480  1420  explorer.exe        x86   0           COYOTE\Beep
 1572  652   spoolsv.exe         x86   0           NT AUTHORITY\SYSTEM
```

Fuente: Ejecución del comando PS. [captura de pantalla].

Ipconfig / ifconfig

```
meterpreter > ipconfig

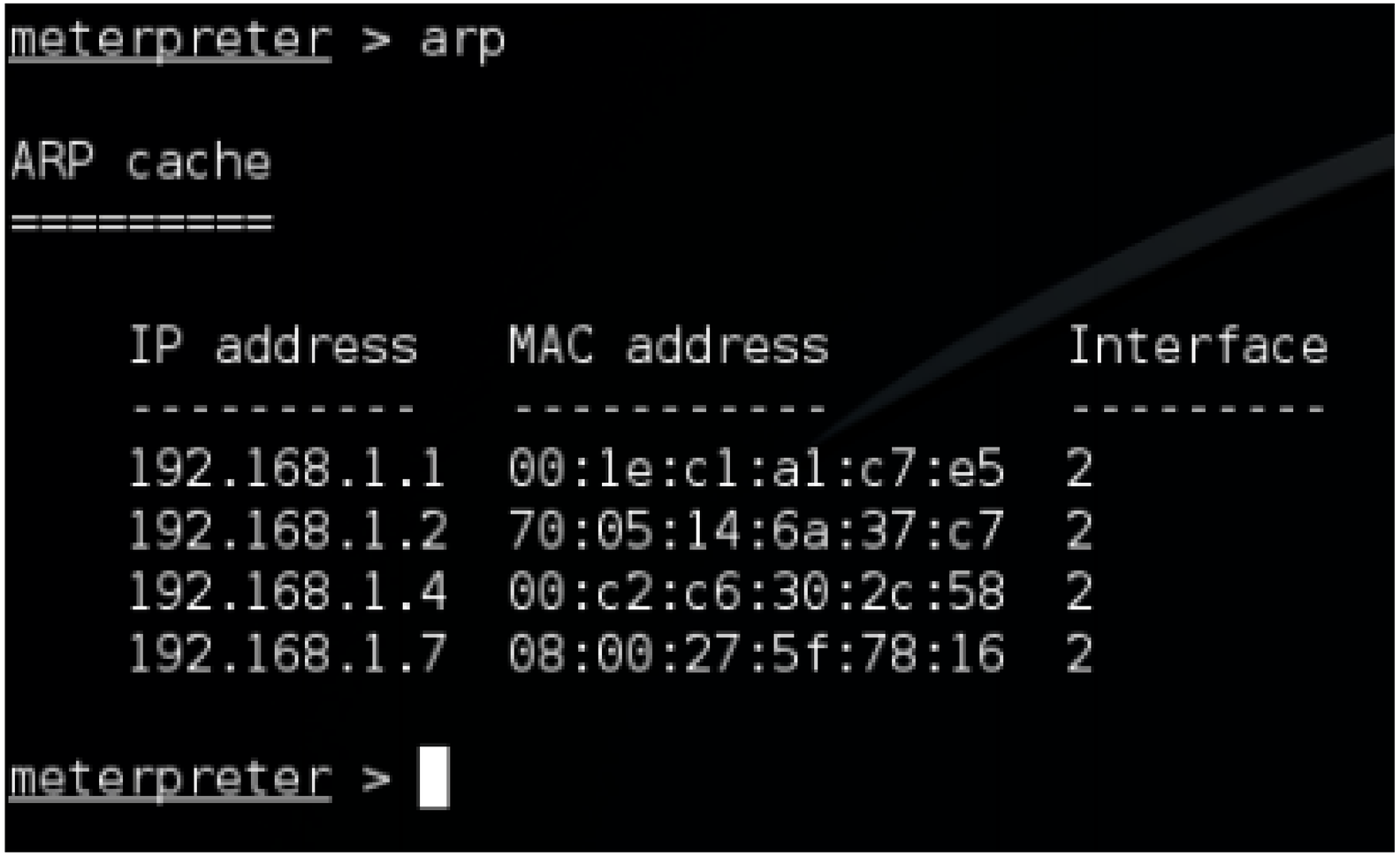
Interface  1
============
Name         : MS TCP Loopback interface
Hardware MAC : 00:00:00:00:00:00
MTU          : 1520
IPv4 Address : 127.0.0.1


Interface  2
============
Name         : AMD PCNET Family PCI Ethernet Adapter
Hardware MAC : 08:00:27:f0:6d:c7
MTU          : 1500
IPv4 Address : 192.168.1.10
IPv4 Netmask : 255.255.255.0

meterpreter >
```

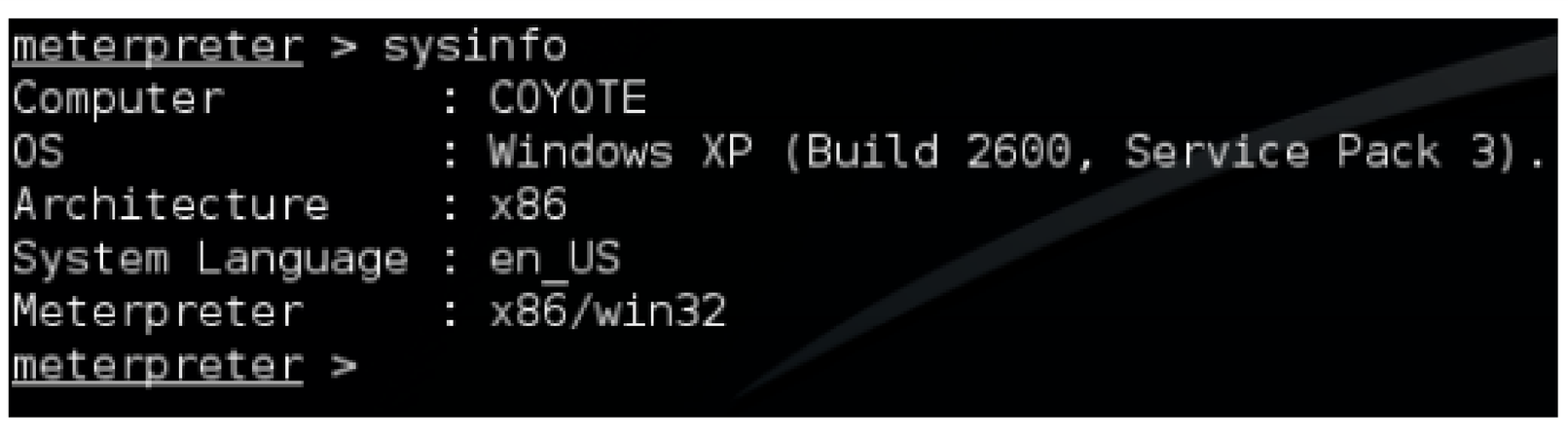
Fuente: Ejecución de ipconfig [captura de pantalla].

Arp



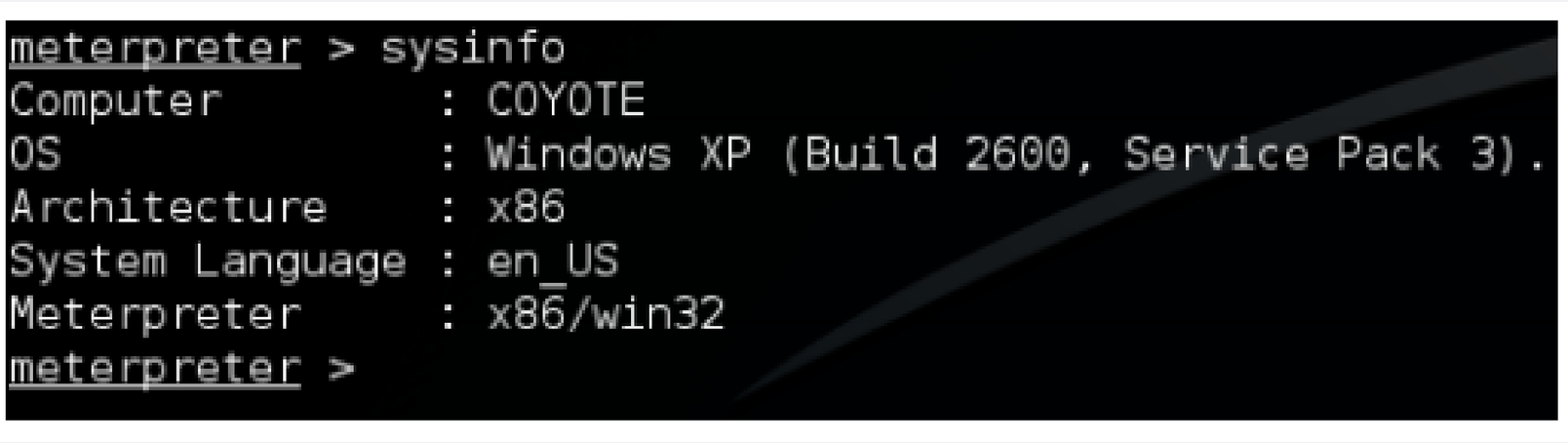
Fuente: Ejecución del comando arp. [captura de pantalla].

Sysinfo



Fuente: Ejecución del comando sysinfo. [captura de pantalla].

Upload / download

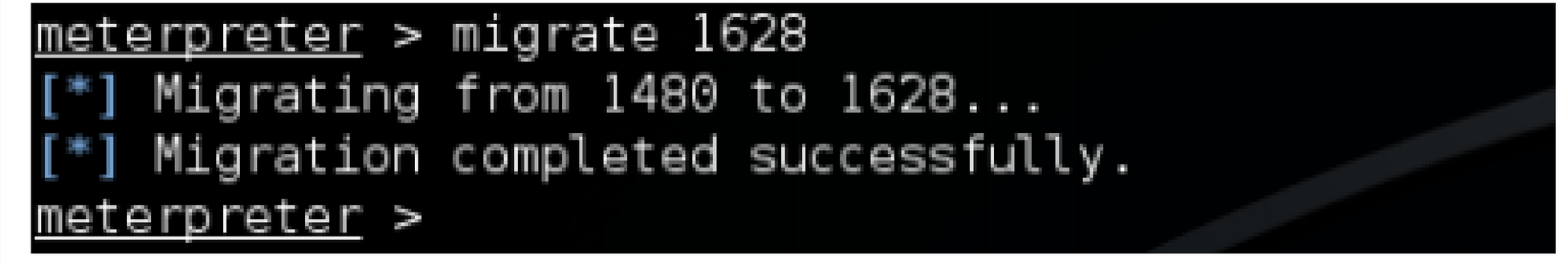


Fuente: ejecución del comando upload del payload cargado. [captura de pantalla].

Migrate

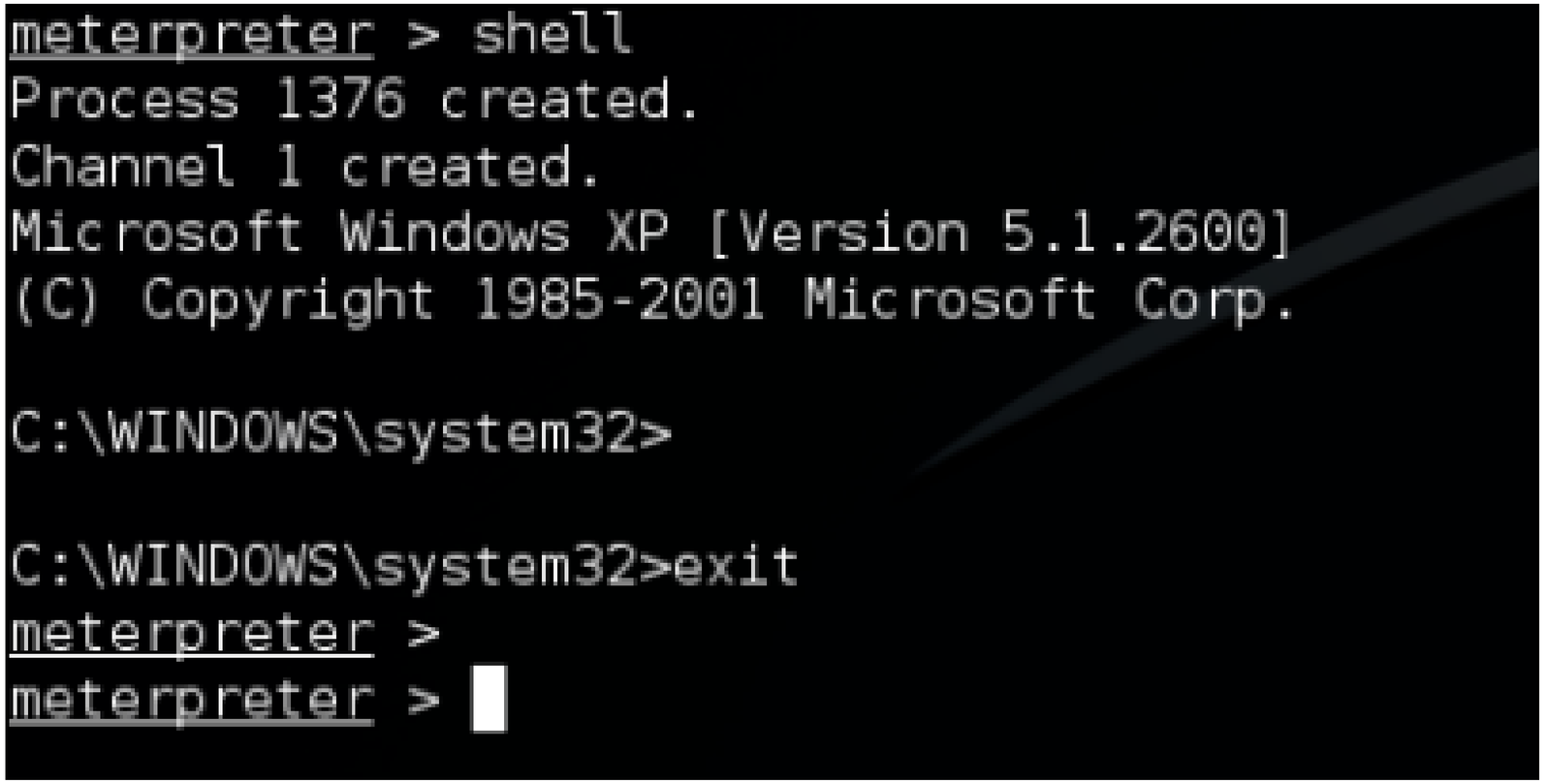
Permite migrar el proceso de Meterpreter (que se ejecuta ‘pegándose’ a otro proceso, para no ser detectado).

Esto se utiliza, generalmente, para migrar a un proceso que no debería ser terminado, en el caso de que actualmente Meterpreter se esté ejecutando dentro de un proceso que posiblemente termine (como un proceso de navegador web).



Fuente: ejecución del comando migrate dentro del payload. [captura de pantalla].

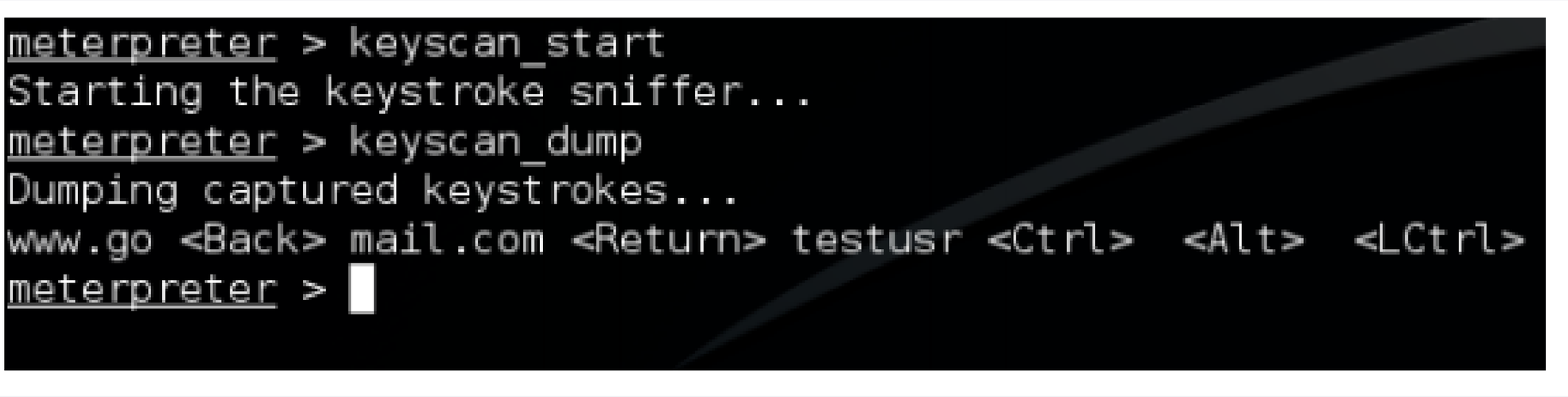
Shell



Fuente: Ejecución del comando Shell dentro del payload. [captura de pantalla].

Keyscan

Inicia un proceso keylogger (captura las teclas presionadas por los usuarios del sistema).



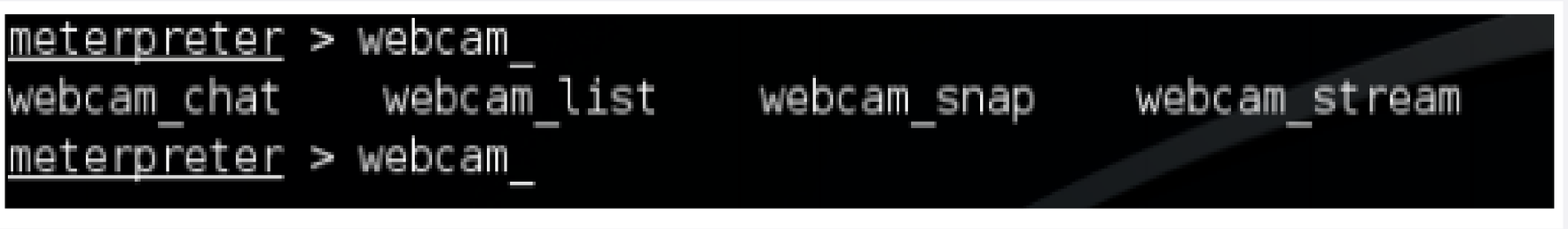
Fuente: Ejecución del comando keyscan del payload cargado. [captura de pantalla].

Screenshot



Fuente: Ejecución del comando screenshot dentro de payload. [captura de pantalla].

Webcam



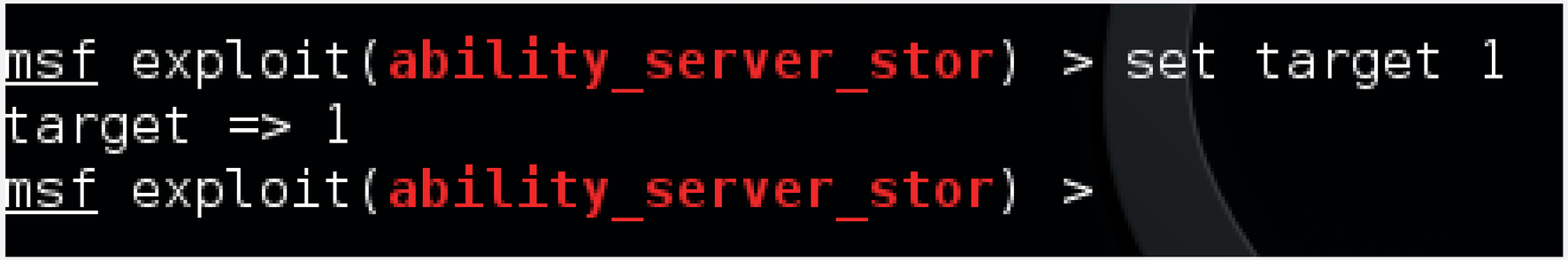
Fuente: Ejecución del comando webcam dentro del payload. [captura de pantalla].

Show targets



Fuente: Ejecución del comando show dentro del payload. [captura de pantalla].

Set target 1



Fuente: Ejecución del comando set target 1 dentro del payload. [captura de pantalla].

set PAYLOAD windows/meterpreter/reverse_tcp

Con Metarpreter (help muestra todo lo que es posible)

comando uictl --> deshabilita mouse o teclado
download --> sube archivos
upload --> baja archivos
cd c:\\ (doble \\ para escapar el comando \)
clearev --> elimina los log de SO atacado
run post/windows/gather/enum_applications -> enumera las app que tiene la PC atacada
keyscan_start / keyscan_dump / keyscan_stop --> keyloguer
sysinfo --> info del SO
ifconfig --> red
arp -->
route --> ruteo
netstat --> sesiones activas

getpid --> el proceso de metarpreter
migrate --> migra el proceso de metarpreter a otro proceso que exista.
netlocalgroup --> lista los grupos a los que pertenece el usuario.
background --> manda la sesión a atrás.
sessions --> selecciono la sesión a la que quiero ir.
screenshot --> captura de pantalla.

timestomp --> permite cambiar fecha y hora de un archivos (ejemplo: timestomp -z "10/03/1995 00:00:00" nombre_archivo)



© Universidad de Palermo
Prohibida la reproducción total o parcial de imágenes y textos.